O que é $bucket? - Classificação por Faixas

- $bucket = agrupa por faixas que VOCÊ define
- Ex: Classificar alunos por nota: 0-5, 5-7, 7-10
- Ex Pokémon: attack 0-50 Fraco, 50-80 Médio, 80-100 Forte, 100-200 Lendário
- Você define boundaries: [0,50,80,100,200]
- Mongo coloca cada doc no balde certo automaticamente!
- Muito usado para classificação por regras!

```
db.pokemon.aggregate([
 { $bucket:{
   groupBy:'$attack',
   boundaries:[0,50,80,100,200],
   default:'Lendário >200'
 }}
])
```

Resultado:
{ _id:0, count:100 } // 0-50
{ _id:50, count:300 } // 50-80
{ _id:80, count:250 } // 80-100

Aula Hoje - $bucket - O Atalho Mágico da Q9 da Prova!

- Na prova Q9: $cond dentro de $cond dentro de $cond... 15 linhas!
- Hoje: $bucket faz MESMA coisa em 3 linhas!
- Vamos aprender $bucket, $bucketAuto e $switch
- Base: Pokémon (800) - já conhecem!
- Objetivo: Classificar Pokémon por poder de forma simples

Prova Q9 (15 linhas):

```
$cond:{ if:{ $lt:['$score',5]}, then:'Excelente',
else:{ $cond:{ if:{ $lt:['$score',15]},
then:'Bom', else:... } } }
```

Hoje $bucket (3 linhas):

```
{ $bucket:{ groupBy:'$score',
boundaries:[0,5,15,30,100] } }
```

Sintaxe Completa do $bucket - boundaries, default, output

- groupBy: campo que vai classificar (ex: $attack)
- boundaries: array com faixas [0,50,80,100,200] - SEMPRE crescente!
- default: onde vai quem não encaixa em nenhuma faixa (opcional mas recomendado)
- output: o que calcular por faixa (total, média, lista de nomes)
- Sem output, só conta quantos caem em cada faixa
- Com output, calcula média, lista, etc!

```
db.pokemon.aggregate([
 { $bucket:{
   groupBy:'$attack',
   boundaries:[0,50,80,100,200],
   default:'Lendário >200',
   output:{
     total:{ $sum:1 },
     mediaHP:{ $avg:'$hp' },
     nomes:{ $push:'$name' }
   }
 }}
])

// total: quantos na faixa
// mediaHP: média de HP da faixa
// nomes: lista de quem está na faixa
```

Exemplo 1 - Pokémon por Ataque

- Vamos classificar 800 Pokémons por ataque:
- 0-50 Fraco, 50-80 Médio, 80-100 Forte, 100-200 Super Forte
- Queremos: total por faixa e 3 exemplos de nomes
- Um único estágio $bucket resolve!
- Compare com Q9 da prova: 15 linhas vs 3 linhas!
- Façam no Compass agora!

```
db.pokemon.aggregate([
 { $bucket:{
   groupBy:'$attack',
   boundaries:[0,50,80,100,200],
   default:'Lendário >200',
   output:{
     total:{ $sum:1 },
     mediaHP:{ $avg:'$hp' },
     exemplos:{ $push:'$name' }
   }
 }}
])

// Resultado esperado:
// _id:0 (Fraco) total:150
// _id:50 (Médio) total:350
// _id:80 (Forte) total:200
```

Exemplo 2 - Pokémon por HP - Outra Faixa!

- Agora por HP: 0-50 Baixo HP, 50-80 Médio HP, 80-255 Alto HP
- Mesma sintaxe, só muda groupBy e boundaries
- Mostra versatilidade do $bucket!
- Pode usar com qualquer campo numérico!
- hp, attack, defense, speed, generation...
- Teste agora com defense!

```
db.pokemon.aggregate([
 { $bucket:{
   groupBy:'$hp',
   boundaries:[0,50,80,255],
   default:'HP Extremo',
   output:{
    total:{ $sum:1 },
    mediaAtaque:{ $avg:'$attack' },
    maisForte:{ $max:'$attack' }
   }
  }}
])
```

$bucketAuto - Mongo decide as faixas sozinho! - Clustering!

- $bucket: VOCÊ define faixas [0,50,80,100]
- $bucketAuto: MONGO define faixas automaticamente!
- Você só diz quantos baldes quer: buckets:3 ou buckets:4
- Mongo divide para ter quantidades parecidas por balde!
- É clustering! K-means simplificado!
- Ex: buckets:3 → 3 grupos: Fraco, Médio, Forte automático!
- Sem precisar pensar nas faixas!

```
$bucket: VOCÊ define
boundaries:[0,50,80,100,200]

$bucketAuto: MONGO define
{ $bucketAuto:{
  groupBy:'$attack',
  buckets:4, // só diz quantos grupos!
  output:{ total:{ $sum:1 } }
}}

Resultado $bucketAuto:
{ _id:{ min:5, max:48 }, total:200 }
{ _id:{ min:48, max:75 }, total:200 }
{ _id:{ min:75, max:100 }, total:200 }
{ _id:{ min:100, max:190 }, total:200 }
```

Exemplo 3 - $bucketAuto - 3 Clusters!

- Vamos dividir Pokémons em 3 clusters por HP automaticamente
- buckets:3 → 3 grupos com quantidades parecidas
- Não precisa definir boundaries!
- Mongo calcula min e max de cada grupo!
- Mostra total, média de ataque por grupo
- Teste com buckets:3, depois buckets:5 - o que muda?

```
db.pokemon.aggregate([
 { $bucketAuto:{
  groupBy:'$hp',
  buckets:3,
  output:{
   total:{ $sum:1 },
   mediaAtaque:{ $avg:'$attack' },
   exemplos:{ $push:'$name' }
  }
 }}
])

// 3 clusters:
// Cluster 1: HP 1-55 (fracos)
// Cluster 2: HP 55-80 (médios)
// Cluster 3: HP 80-255 (fortes)
// Quantidades parecidas!
```

Algoritmo de Classificação por Regras com $switch - Alternativa ao $cond

- Na prova usamos $cond aninhado - funciona mas fica gigante
- $switch é IF/ELSE limpo, igual programação!
- Sintaxe: branches com case e then, default no final
- Mesmo resultado do $bucket e do $cond, mas mais legível!
- Use $addFields ou $project com $switch
- Depois pode $group por classe criada!
- Vamos comparar os 3 jeitos!

```
db.pokemon.aggregate([
 { $addFields:{
  classe:{
   $switch:{
    branches:[
     { case:{ $lt:['$attack',50]},
then:'Fraco' },
     { case:{ $lt:['$attack',80]},
then:'Médio' },
     { case:{ $lt:['$attack',100]},
then:'Forte' }
    ],
    default:'Lendário'
   }
  }
 }},
 { $group:{ _id:'$classe', total:{ $sum:1 } } }
])
```

Comparação - 3 jeitos de classificar - Qual prefere?

- Q9 prova $cond aninhado: 15 linhas, difícil de ler
- $switch: 10 linhas, parece IF/ELSE, legível!
- $bucket: 3 linhas, MAIS FÁCIL, só define faixas!
- Os 3 dão mesmo resultado!
- Para prova: use $cond (cobrado)
- Para vida real: use $bucket ou $switch!
- Hoje vamos usar $bucket e $switch - mais simples!
- Qual vocês acham mais fácil?

```
// $cond (prova)
$cond:{ if:{ $lt:['$a',50]}, then:'Fraco',
else:{ $cond:{...} } }

// $switch (limpo)
$switch:{ branches:[{case:{ $lt:['$a',50]},
then:'Fraco'}], default:'Forte' }

// $bucket (MAIS FÁCIL)
{ $bucket:{ groupBy:'$a',
boundaries:[0,50,80,100] } }
```

Lab Hoje - 5 Exercícios Só de $bucket e $switch

- Ex1 - $bucket por ataque [0,50,80,100,200]
- Ex2 - $bucket por HP [0,50,80,255]
- Ex3 - $bucketAuto 3 clusters por attack
- Ex4 - $switch classificando Fraco/Médio/Forte/Lendário + $group
- Ex5 - LoL - $bucket por vida [0,550,600,650,1000] + $bucketAuto por dano
- Tudo que já viram + $bucket novo!